ARQUITECTURA DE AGENTES IA · UTDT 2026

Módulo 2
Diseño de Sistemas Agénticos

SDLC · Design Principles · Task Suitability · ADK2

UTDT 2026 | MÓDULO 2 DE 13

¿Qué vamos a aprender hoy?

FUNDAMENTOS

- Componentes centrales de un sistema agéntico
- Criterios para seleccionar el modelo correcto
- Tipos de herramientas (tools) disponibles
- Memoria: corto y largo plazo

DISEÑO & PRODUCCIÓN

- Trade-offs: velocidad, costo, escalabilidad
- Single-agent vs. multiagente
- Diseño iterativo y evaluación continua
- ADK2 y GCP Agent Platform en la práctica

> *El buen diseño de agentes comienza estrecho y medible, luego expande.*

Los 4 Componentes Centrales

FOUNDATION MODEL

- Razonamiento
- Comprensión
- Generación

TOOLS

- Funciones locales
- APIs externas
- MCP servers

MEMORY

- Contexto en sesión
- Bases de datos
- Knowledge graphs

ORCHESTRATION

- Composición
- Scheduling
- Supervisión

Cada componente se diseña de forma independiente y se compone de manera deliberada.

PRINCIPIO FUNDAMENTAL

Empezar Estrecho, Expandir Iterativamente

> *Automatizar "toda la atención al cliente" → ahogarse en casos borde.*
> *Empezar con* `cancel_order` *→ probar valor → expandir.*

ANTI-PATRÓN

- Over-scoping: demasiados casos desde el día 1
- Under-scoping: una sola acción, ignora el resto
- Objetivos vagos: "mejorar satisfacción" sin métricas

BUENA PRÁCTICA

- Una tarea, una métrica de éxito clara
- MVP en producción lo antes posible
- Feedback loop desde el primer día

Dimensión 1: Complejidad de la Tarea

TAREAS ABIERTAS → MODELOS GRANDES

- Razonamiento multi-paso
- Ambigüedad semántica alta
- Requiere creatividad o síntesis
- *Ejemplo: análisis de documentos legales complejos*

TAREAS ESTRUCTURADAS → MODELOS PEQUEÑOS

- Clasificación, extracción, formateo
- Salidas predecibles y delimitadas
- Latencia crítica
- *Ejemplo: clasificar ticket de soporte*

Dimensión 2 y 3: Modalidad y Apertura

MODALIDAD

- Solo texto: mayoría de workflows empresariales
- Multimodal: imagen, audio, video
- GPT-5, Claude Opus 4.1 → multimodal nativo
- Gemini 2.5 Pro → nativo en GCP / Vertex AI

APERTURA

- Open-weight: Llama 3, DeepSeek → self-hosted, privacidad, costo
- API propietaria: GPT-5, Claude → rendimiento máximo, menor control
- Decisión: compliance + latencia + presupuesto

Dimensiones 4–6: Fine-tuning, Costo y Benchmarks

- Pretrained vs. custom-trained: modelos de propósito general vs. fine-tuning en dominio específico (medicina, derecho, código interno)
- Costo/latencia: *dynamic model routing* — modelo barato para rutinas, modelo caro para complejidad
- Task specificity: HELM y MMLU como línea de base comparativa

> *No existe "el mejor modelo". Existe el modelo correcto para cada eje de decisión.*

Dynamic routing en ADK2: un router lightweight evalúa la query y despacha al modelo apropiado, implementado como agente/función custom que instancia el `LlmAgent` con el modelo elegido según la query. Vertex AI también ofrece Model Optimizer como endpoint gestionado con el mismo objetivo — la versión custom da control total sobre el criterio de routing (y es la que enseña el lab); Model Optimizer ahorra esa lógica a cambio de menos control.

REFERENCIA · MODELOS OPEN-WEIGHT

Comparativa: Modelos de Peso Abierto

MODELO	MMLU	PARÁMETROS	VRAM	HARDWARE
Gemma 2 (Google)	72.1	9B	22.5 GB	RTX 3090
Phi-3 (Microsoft)	77.5	14.7B	29.4 GB	A100
Llama 3 (Meta)	79.3	70B	160 GB	4×A100

MMLU = Massive Multitask Language Understanding — benchmark de referencia académico.

- Gemma 2 es la opción nativa en GCP / Vertex AI para auto-hosting
- Llama 3 70B requiere infraestructura multi-GPU significativa
- Self-hosting implica: costo de hardware + equipo MLOps + latencia de actualización

Comparativa: Modelos vía API

MODELO	MMLU	COSTO INPUT ×LLAMA 3.1	COSTO OUTPUT ×LLAMA 3.1
Llama 3.1 405B	84.5	1× (base)	1× (base)
o4-mini (OpenAI)	83.2	5.5×	7.3×
Gemini 2.5 Pro	86.2	12.5×	25×
Claude Opus 4.1	86.5	75×	125×

Precios relativos a Llama 3.1 @ $0.20/$0.60 por millón de tokens.

> *Performance marginal del 2–4% entre modelos top puede costar 75–125× más. Cuantificar el ROI antes de elegir.*

TRADE-OFFS DE DISEÑO

Los 4 Ejes de Decisión Arquitectónica

VELOCIDAD VS. PRECISIÓN

- Tiempo real → velocidad
- Legal/médico → precisión

ESCALABILIDAD

- GPU allocation dinámica
- Elastic provisioning
- Async task execution

CONFIABILIDAD

- Fault tolerance
- Redundancia
- Monitoring + feedback loops

COSTO

- Dev cost + operational cost
- vs. valor entregado (ROI)

Cuando el Tiempo Real No Es Negociable

ALTA VELOCIDAD

- Chatbots conversacionales
- Detección de fraude en tiempo real
- Sistemas de recomendación

ALTA PRECISIÓN

- Diagnóstico médico asistido
- Análisis de contratos legales
- Auditorías financieras automatizadas

Fraude no es "error tolerable": falso negativo = pérdida directa; falso positivo bloquea a un cliente legítimo. Patrón real, dos capas: un clasificador de milisegundos decide el flujo, y detrás un modelo más preciso —o un analista— revisa la franja de decisión.

En ADK2 (2.8.0+): `timeout` *y* `on_model_error_callback` *son campos de* `LlmAgent` *— dos mecanismos independientes, no una cadena: uno corta por tiempo, el otro reacciona a errores del modelo.*

`timeout` y `on_model_error_callback`: Dos Mecanismos Distintos

```python
# timeout: acota el turno completo (todas las llamadas + tools), no una
# sola inferencia. Si se excede, propaga NodeTimeoutError — no pasa por
# ningún callback; hay que capturarlo alrededor del Runner.
agente_rapido = LlmAgent(name="triage", model="gemini-2.5-flash-lite", timeout=2.0)

# on_model_error_callback: se dispara cuando la LLAMADA AL MODELO lanza
# una excepción (ej. error de la API) — no por timeout.
def usar_modelo_secundario(callback_context, llm_request, error):
    print(f"gemini-2.5-pro falló ({error}); habría que reintentar con flash")
    return None  # None = dejar que el error se propague

agente_preciso = LlmAgent(
    name="detector_fraude", model="gemini-2.5-pro",
    on_model_error_callback=usar_modelo_secundario,
)
```

> No se combinan automáticamente en un failover: un timeout no dispara el callback. Devolver un `LlmResponse` desde el callback sustituye la respuesta rota; `None` deja que el error se propague.

TRADE-OFF · ESCALABILIDAD

Escalado Dinámico de Recursos

- Problema: los agentes tienen cargas de trabajo variables e impredecibles
- Solución: infraestructura elástica que escala automáticamente con la demanda

> Hybrid cloud scaling en GCP: GPU base on-prem + cloud burst en Vertex AI para picos de demanda. Cloud Run jobs para tareas async de agentes.

- GPU allocation dinámica: asignar compute solo cuando el agente está activo
- Elastic provisioning: crecer y decrecer según demanda sin intervención manual
- Async task execution: desacoplar la solicitud de la respuesta → mayor throughput

Anti-patrón: static resource allocation — GPUs ociosas o sobrecargadas sin ajuste dinámico.

Fault Tolerance y Redundancia

> *Un agente no probado es un agente no confiable.*

- Fault tolerance: detectar fallas y recuperarse sin colapsar el sistema
- Redundancia: múltiples rutas de ejecución disponibles para tareas críticas
- Monitoring + feedback loops: métricas en tiempo real, alertas, rollback automático

En ADK2

- `retry_config` (`RetryConfig` de `google.adk.workflow`) con backoff exponencial
- `on_model_error_callback` / `on_tool_error_callback` para degradación elegante
- Integration con Cloud Monitoring (GCP)

Patrones Comunes

- Circuit breaker para APIs externas
- Dead letter queues para tareas fallidas
- Health checks periódicos

`retry_config`: Código Mínimo

```python
from google.adk.agents import LlmAgent
from google.adk.workflow import RetryConfig

agente = LlmAgent(
    name="agente_reclamos",
    model="gemini-2.5-flash",
    retry_config=RetryConfig(
        max_attempts=3,        # default: 5 intentos en total
        initial_delay=1.0,     # segundos antes del primer reintento
        backoff_factor=2.0,    # cada intento espera 2× más que el anterior
        exceptions=["ServerError"],  # solo reintenta estos errores
    ),
)
```

`retry_config` resuelve cuántas veces y con qué espera reintentar la misma llamada. Qué hacer cuando se agotan los reintentos sigue siendo responsabilidad de `on_model_error_callback`.

Las 3 Categorías de Herramientas

TIPO	MECANISMO	EJEMPLO
Local	Función interna, pura, sin I/O externo	Cálculo matemático, lookup en DB local
API-based	Llamada HTTP a servicio externo	Weather API, precios de bolsa
MCP	Protocolo estándar cliente-servidor (JSON-RPC 2.0) para exponer tools/contexto	Perfil de usuario, estado de conversación

MCP = Model Context Protocol — estándar de Anthropic para la comunicación agente-herramienta ("USB-C para IA"); no es un mecanismo de inyección directa en el razonamiento del modelo, sino un protocolo de transporte que estandariza cómo un agente descubre e invoca tools y fuentes de contexto expuestas por un servidor MCP.

- Las tools locales son deterministas y rápidas → usar para operaciones frecuentes
- Las tools API-based introducen latencia y dependencia → diseñar con timeouts y fallbacks
- Las tools MCP enriquecen el contexto disponible sin aumentar los prompts manualmente

Memoria de Corto Plazo

- Definición: ventana de contexto activa durante la sesión actual
- Función: coherencia conversacional, acceso a intercambios recientes
- Límite: ventana de contexto del modelo (tokens máximos)

```python
# ADK2: la sesión se inyecta en el Runner, no en el agente
from google.adk.agents import LlmAgent
from google.adk.runners import Runner
from google.adk.sessions import InMemorySessionService

agent = LlmAgent(name="agente_soporte", model="gemini-2.5-flash")

runner = Runner(
    agent=agent,
    app_name="soporte",
    session_service=InMemorySessionService(),
    # El Runner persiste el estado de la sesión durante toda la conversación
)
```

> *El contexto en sesión es volátil — no persiste entre llamadas independientes.*

Memoria de Largo Plazo

- Definición: almacenamiento persistente entre sesiones y usuarios
- Implementaciones posibles:
  - Bases de datos vectoriales (Vertex AI Vector Search, Pinecone)
  - Knowledge graphs (estructurar relaciones entre entidades)
  - Modelos fine-tuned (conocimiento implícito en pesos)

> En GCP: Firestore para estado de agentes, Vertex AI Vector Search para RAG, BigQuery para histórico analítico. ADK2 provee `VertexAiMemoryBankService`, que extrae memoria de largo plazo automáticamente de las sesiones cerradas.

- La memoria de largo plazo habilita personalización y aprendizaje organizacional
- Requiere estrategias de actualización, expiración y control de acceso

Memory Bank: Código Mínimo

```python
from google.adk.memory import VertexAiMemoryBankService
from google.adk.tools import load_memory

# El agente puede buscar en la memoria de largo plazo del usuario:
agente = LlmAgent(name="agente_soporte", model="gemini-2.5-flash", tools=[load_memory])

runner = Runner(
    agent=agente, app_name="soporte", session_service=session_service,
    memory_service=VertexAiMemoryBankService(
        project="sandbox-ai-zabaljauregui", location="us-central1",
        agent_engine_id="1234567890",
    ),
)

# Al cerrar una sesión, se ingiere a la memoria de largo plazo:
await runner.memory_service.add_session_to_memory(session)
```

`memory_service` es un parámetro del `Runner` distinto de `session_service` — uno persiste el turno actual, el otro extrae y busca memoria entre sesiones. `load_memory` es el tool nativo que consulta esa memoria por texto libre.

El Cerebro del Sistema Agéntico

Orchestration = lógica que compone, programa y supervisa secuencias de habilidades multi-paso.

- Composición: qué herramientas/agentes invocar y en qué orden
- Scheduling: cuándo ejecutar cada paso (secuencial, paralelo, condicional)
- Supervisión: monitorear resultados y decidir si continuar, reintentar o escalar

```
# ADK2: orquestación declarativa con SequentialAgent
from google.adk.agents import SequentialAgent, LlmAgent

pipeline = SequentialAgent(
    name="soporte_pipeline",
    sub_agents=[
        clasificador_agent,    # Paso 1: clasificar intención
        enriquecedor_agent,    # Paso 2: buscar contexto
        respuesta_agent,       # Paso 3: generar respuesta
    ]
)
```

Single-Agent: Simplicidad y Alcance Acotado

CUÁNDO USAR

- Tarea única y bien delimitada
- No hay paralelismo natural
- El overhead de coordinación no vale la pena
- *Chatbots, code gen, task automation*

VENTAJAS

- Debugging simple y predecible
- Menor consumo de tokens
- Deployment y operación sencillos
- Latencia mínima (sin coordinación)

En ADK2: `LlmAgent` *es la unidad single-agent por defecto. Empezar siempre aquí.*

Multiagente: Colaboración y Especialización

BENEFICIOS

- Paralelismo: múltiples tareas simultáneas
- Especialización: cada agente optimizado para su rol
- Resiliencia: fallo de un agente no colapsa el sistema
- Escalabilidad horizontal

COSTOS

- Complejidad de coordinación
- Mayor consumo de tokens
- Debugging distribuido complejo
- Latencia de red entre agentes

Multiagente en ADK2 y GCP

```python
# ADK2: sistema multiagente con ParallelAgent
from google.adk.agents import ParallelAgent, LlmAgent

# Agentes especializados en paralelo
analisis_paralelo = ParallelAgent(
    name="analisis_multidimensional",
    sub_agents=[
        LlmAgent(name="analista_sentimiento", ...),
        LlmAgent(name="extractor_entidades", ...),
        LlmAgent(name="clasificador_urgencia", ...),
    ]
)
```

GCP Agent Platform: permite deploy de cada sub-agente como Cloud Run service independiente, con auto-scaling por agente y comunicación via Pub/Sub o gRPC.

DYNAMIC MODEL ROUTING

Enrutar al Modelo Correcto en Tiempo Real

> *Route simple queries to cheap small model; complex queries to large model.*

```python
# `model` de LlmAgent es estático tras instanciar → el router
# re-instancia el agente en cada request, no reasigna `.model`.
def elegir_modelo(query: str) -> str:
    if es_simple(query):  return "gemini-2.5-flash-lite"
    if es_media(query):   return "gemini-2.5-flash"
    return "gemini-2.5-pro"

def crear_agente_para_query(query: str) -> LlmAgent:
    return LlmAgent(name="agente_enrutado", model=elegir_modelo(query), instruction="...")

agente = crear_agente_para_query(query_del_usuario)  # un LlmAgent nuevo por request
```

El ahorro no es una constante del patrón: depende de tu mezcla de tráfico. La cuenta, en la slide siguiente.

¿Cuánto Ahorra Realmente el Routing?

No hay un porcentaje de catálogo. Hay una fórmula, y una variable que casi todos miden mal:

$$costo_medio = \Sigma_i \left(share\ de\ tokens\ del\ nivel\ i \times precio\ del\ modelo\ i \right) + costo\ del\ clasificador$$

LO QUE PONDERA

- No la cantidad de queries baratas
- Sí su share de tokens
- Las queries complejas son también las largas: pocas, pero pesadas

MISMO ROUTER, DOS RESULTADOS

- Corpus del lab (2 de 5 complejas, ~73% de los tokens) → ~24%
- Mezcla 80/15/5 con tokens parejos → ~87%

Los dos números salen de aplicar las fórmulas de costo del Ejercicio 3 a dos mezclas distintas — no de un benchmark de proveedor. El punto no es el número: es que el ahorro es una propiedad de tu tráfico, no del patrón. Antes de comprometerlo en un comité, medí tu mezcla.

DISEÑO ITERATIVO

El Ciclo de Vida del Desarrollo Agéntico

1. MVP

Una tarea, un modelo, herramientas mínimas

→

2. Test

Funcional + UX + human-in-the-loop

→

3. Feedback

KPIs, señales implícitas/explícitas

→

4. Refinar

Ampliar scope, mejorar tools, ajustar modelo

El ciclo se repite hasta alcanzar los objetivos de producción.

Evaluar Temprano y Frecuentemente

EVALUACIÓN FUNCIONAL

- Correctness: ¿la respuesta es correcta?
- Boundary testing: casos extremos y adversariales
- Domain metrics: precisión, recall, F1
- Tool recall: ¿se usó la herramienta correcta?
- Parameter accuracy: ¿con los parámetros correctos?

EVALUACIÓN UX

- NPS: Net Promoter Score
- Task completion rate
- Señales explícitas: thumbs up/down
- Señales implícitas: abandono, reformulación
- Human-in-the-loop para casos complejos

El Rol del Humano en el Ciclo de Evaluación

> *Un agente no probado es un agente no confiable.*

- Human-in-the-loop (HITL): revisión humana de decisiones del agente en puntos críticos
- No implica intervención en cada paso — implica *checkpoints estratégicos*

Cuándo activar HITL

- Decisiones de alto impacto
- Confianza del modelo baja
- Primeras iteraciones del agente

En ADK2

- `LongRunningFunctionTool` pausa la tool call hasta aprobación
- `before_tool_callback` como gate de revisión, antes de ejecutar
- Audit trails vía callbacks custom

HITL en ADK2: Dos Mecanismos, Dos Snippets

```python
from google.adk.tools import LongRunningFunctionTool

def aprobar_liquidacion(reclamo_id: str, monto: float) -> dict:
    """Pausa la tool call hasta que un supervisor apruebe o rechace."""
    ticket_id = crear_ticket_aprobacion(reclamo_id, monto)
    return {"status": "pending", "ticket_id": ticket_id}

agente = LlmAgent(..., tools=[LongRunningFunctionTool(func=aprobar_liquidacion)])
```

```python
# before_tool_callback: bloquea la tool call ANTES de ejecutarla
def gate_liquidacion(tool, args, tool_context):
    if args["monto"] > UMBRAL_APROBACION_HUMANA:
        return {"exito": False, "requiere_aprobacion_humana": True}
    return None  # None = dejar pasar la llamada original
```

Gobernanza del Agente en Entorno Regulado

El HITL de la slide anterior es un mecanismo. La gobernanza es quién responde por él. Cinco preguntas que un regulador hace y que la arquitectura tiene que poder contestar sin reconstruir nada a mano:

PREGUNTA	ARTEFACTO QUE LA CONTESTA	DÓNDE VIVE
¿Qué se loguea?	Prompt, tools invocadas con sus parámetros, salida, modelo y versión, latencia, costo, decisión del gate	Cloud Logging → BigQuery (inmutable, append-only)
¿Quién aprueba el HITL gate?	Rol y persona, no "el sistema": aprobador nombrado, con monto/riesgo máximo delegado y suplente	Matriz de delegación firmada, fuera del código
¿Dónde vive la PII?	Lo que entra al prompt es tan sensible como la base de origen	VPC-SC / red privada, con data residency y tokenización previa
¿Cuánto se retiene el audit trail?	El plazo lo fija la norma sectorial, no el equipo	Retención declarada y probada con una restauración real
¿Cómo se versiona el modelo?	Modelo + prompt + tools + umbrales son una sola versión, no cuatro	Un tag por release, con el snapshot de evaluación al lado

> El proveedor puede actualizar el modelo por debajo sin avisar. Si el log no guarda la versión exacta que respondió, un caso de hace ocho meses es irreproducible — y esa es, literalmente, la primera pregunta de un examen.

ROLLOUT EN PRODUCCIÓN

Despliegue Progresivo y Monitoreo

- Phased rollout: no desplegar al 100% desde el día 1

Fase 1 — Canary (1-5%)

- Usuarios internos o beta testers
- Validar comportamiento en producción real

Fase 2 — Ramp Up (10-50%)

- Monitorear KPIs continuamente
- Recopilar feedback a escala

Fase 3 — Full (100%)

- Solo si KPIs son satisfactorios
- Mantener capacidad de rollback

> *GCP: Cloud Deploy soporta canary deployments nativos para servicios de agentes.*

ANTI-PATRONES

Los 5 Errores Más Comunes en Diseño de Agentes

#	ANTI-PATRÓN	CONSECUENCIA
1	Over-scoping inicial	Edge cases infinitos, imposible de probar
2	Under-scoping	El agente no cubre los casos de mayor volumen
3	Objetivos vagos	Sin criterio de éxito → nunca se sabe si funcionó
4	Agente no probado	Comportamiento imprevisible en producción
5	Static resource allocation	GPUs ociosas o sistema saturado sin aviso

> *Cada uno de estos errores cuesta más corregirlo tarde que prevenirlo temprano.*

ARQUITECTURA · GCP

GCP Agent Platform: Piezas Clave

Vertex AI

- Model serving (Gemini 2.5 Pro)
- Vector Search para RAG
- Model Garden para selección de modelos

Cloud Run

- Deploy de agentes como servicios
- Auto-scaling por agente
- Jobs para tareas async

Firestore + Pub/Sub

- Estado persistente de sesiones
- Comunicación entre agentes
- Event-driven orchestration

Google Agent Development Kit 2: Filosofía

> *ADK2 es el framework de Google para construir, evaluar y desplegar agentes en producción.*

- Filosofía: composición de agentes como primitivas de primera clase
- Integración nativa con Vertex AI, Gemini, Cloud Run, Firestore
- Patrones soportados: single-agent, `SequentialAgent`, `ParallelAgent`, `LoopAgent` (hasta `max_iterations`), custom

```python
from google.adk.agents import LlmAgent
from google.adk.tools import FunctionTool

def consultar_reclamo(reclamo_id: str) -> dict:
    """Consulta el estado de un reclamo por su ID."""
    return {"status": "en_revision", "reclamo_id": reclamo_id}

agente = LlmAgent(
    name="agente_reclamos", model="gemini-2.5-flash",
    tools=[FunctionTool(consultar_reclamo)],
    instruction="Gestionás reclamos de tarjeta y seguros de clientes.",
)
```

Evaluation Framework en ADK2

```python
# Evaluación funcional con ADK2 — pytest + AgentEvaluator
import pytest
from google.adk.evaluation.agent_evaluator import AgentEvaluator

@pytest.mark.asyncio
async def test_agente_soporte_eval():
    await AgentEvaluator.evaluate(
        agent_module="soporte.agent",              # módulo con `root_agent`
        eval_dataset_file_path_or_dir="tests/eval/soporte.test.json",
        num_runs=2,
    )
    # evaluate() corre el agente real (no mocks) y hace el assert
    # internamente contra los umbrales de tests/eval/test_config.json
```

`AgentEvaluator.evaluate()` es un método estático y asíncrono. Sus criterios nativos son `tool_trajectory_avg_score` y `response_match_score` (configurados en `test_config.json`) — no una lista de métricas como parámetro.

SÍNTESIS

Los 5 Takeaways del Módulo 2

1. Empezar estrecho y medible: una tarea (ej: `cancel_order`) → demostrar valor → expandir.
2. Evaluar temprano y frecuentemente: tool recall, parameter accuracy, task completion rates.
3. Single-agent = simplicidad + alcance acotado; multiagente = colaboración + costo de coordinación.
4. Selección de modelo = decisión de 6 ejes; revisar a medida que evolucionan las necesidades.
5. Diseño iterativo + phased rollout = sistemas confiables en producción.

Hoja de Decisión para Líderes

Los cinco takeaways anteriores son principios de ingeniería. Esto es lo que se firma.

BUILD VS. BUY

- Buy si el proceso es genérico y el diferencial no es tuyo
- Build si el valor está en *tus* datos o reglas
- Lo que nunca se compra: el corpus de evaluación

CUÁNDO NO CONVIENE UN AGENTE

- Proceso determinista que ya funciona → es un workflow
- Costo del error > ahorro y sin gate humano viable
- No hay forma de medir el éxito con un número
- El volumen no paga ni la evaluación

QUIÉN FIRMA EL RIESGO

- El umbral de un gate es una declaración de riesgo aceptado, no un parámetro técnico
- Lo firma el dueño del proceso de negocio; ingeniería lo implementa
- Si nadie lo firma, el default es quien hizo el deploy

QUÉ KPI SE COMPROMETE

- Uno solo, medible antes del piloto (línea de base)
- Costo por interacción y tasa de escalación a humano sobreviven al comité
- Sin línea de base previa no hay con qué comparar

CONEXIONES · MAPA DEL CURSO

Este Módulo en el Contexto del Curso

← MÓDULO 1

Decisión workflow vs. agente → alimenta el scoping de este módulo

→ MÓDULO 4

Tool Use: tipos local/API/MCP introducidos aquí, deep-dive allá

→ MÓDULO 5

Orchestration: introducida aquí, tratamiento completo allá

→ MÓDULOS 6, 8, 9

Memory, Multiagente, Evaluación: introducidos hoy, desarrollados luego

MÓDULO 2 — CIERRE

Preguntas & Discusión

Próximo: Módulo 3 → Fundamentos de LLMs para Agentes

Lab práctico: completar los TODO de *lab.md* en el repositorio del curso (la solución de referencia, *lab_solucion.md*, es material exclusivo del docente)

ADK2 | GCP AGENT PLATFORM | UTDT 2026